Part 1 | Multiple Choice (20 questions)

The questions are set in real data-analysis contexts: the Titanic survival study, iris flowers, Palmer penguins, abalone, wine quality, car road tests and adult income. All excerpts are simplified from real public datasets for printing.

1. A wine quality study records ratings from 0 to 10. The script keeps only the ratings above 5. What is printed?

```
ratings = [5, 6, 6, 7, 5, 8]
above = [r for r in ratings if r > 5]
print(len(above))
```

(A) 2

(B) 3

(C) 4

(D) 6

2. A biologist counts penguins by species with collections.Counter. What is printed?

```
from collections import Counter
species = ["Adelie", "Gentoo", "Adelie", "Chinstrap", "Gentoo",
"Gentoo"]
c = Counter(species)
print(c["Adelie"])
```

(A) 3

(B) 1

(C) 4

(D) 2

3. A shell ring file stores each ring count as a string ("7", "12", ...). rings = ["7", "12", "9"]. Which line computes the total as an integer?

(A) `sum(rings)`

(B) `sum(int(r) for r in rings)`

(C) `int(sum(rings))`

(D) `sum(map(str, rings))`

4. A script pairs car names with their mpg values using zip. What is printed?

```
cars = ["Mazda RX4", "Datsun 710"]
mpg = [21.0, 22.8]
print(" | ".join(f"{c}: {m}" for c, m in zip(cars, mpg)))
```

(A) `"Mazda RX4: 21.0 | Datsun 710: 22.8"`

(B) `"Mazda RX4, Datsun 710"`

(C) `"21.0 | 22.8"`

(D) `["Mazda RX4: 21.0", "Datsun 710: 22.8"]`

5. A team classifies wind speeds by band. What is printed by print(band(800))?

```
def band(v):
    if v < 500:
        return "low"
```

```
    elif v < 1000:
        return "mid"
    else:
        return "high"
print(band(800))
```

(A) "low"

(B) "mid"

(C) "high"

(D) 800

6. A botanist measures petal lengths (cm) of 5 iris flowers: petal = np.array([1.4, 4.7, 5.1, 1.3, 5.9]). What is printed?

```
print(np.round(petal.mean(), 2))
```

(A) 3.5

(B) 18.4

(C) 5.9

(D) 3.68

7. Wave heights (m) are logged in an array. How many observations exceed 0.3 m?

```
heights = np.array([0.35, 0.53, 0.0, 0.42, 0.25])
print((heights > 0.3).sum())
```

(A) 2

(B) 3

(C) 4

(D) 5

8. A team needs the three largest mpg values from mpg = np.array([21.0, 21.0, 22.8, 18.7, 18.1, 14.3]). Which line gives them?

(A) np.max(mpg, 3)

(B) np.sort(mpg)[:3]

(C) np.sort(mpg)[-3:]

(D) np.unique(mpg)[3:]

9. A 12-element sensor array is reshaped into a table with 3 rows. What is printed?

```
x = np.arange(12)
print(x.reshape(3, 4).shape)
```

(A) (3, 4)

(B) (4, 3)

(C) (12,)

(D) (1, 12)

10. To reproduce a teaching demo, the team samples 5 penguin indices from a pool of 344 with a fixed random seed. Which line gives the same 5 indices on every run?

(A) np.random.choice(5, size=5)

(B) np.random.default_rng(seed=42).choice(344, size=5, replace=False)

(C) random.sample(344, 5)

(D) np.random.choice(344, 5, replace=True)

Context for Q11-Q14: the team works with the Palmer Penguins DataFrame loaded below.

```
df = pd.read_csv("penguins.csv")   # one row per penguin in the Palmer
Archipelago (344 rows)

   species  island      flipper_length_mm  body_mass_g
   Adelie   Torgersen   181                3750
   Adelie   Torgersen   186                3800
   Adelie   Torgersen   195                3250
   Adelie   Torgersen   NaN                NaN
   Adelie   Torgersen   193                3450
```

Field meanings: species = penguin species (Adelie, Chinstrap, Gentoo); island = island in the Palmer Archipelago (Torgersen, Biscoe, Dream); flipper_length_mm = flipper length in millimetres; body_mass_g = body mass in grams. The full file (344 rows) also contains bill_length_mm and bill_depth_mm.

11. Continuing with the penguins context above, the full DataFrame has 344 rows. What does df["species"].value_counts() return?

(A) A DataFrame with one column per species

(B) The number 344

(C) A Series with the count per species (Adelie 152, Gentoo 124, Chinstrap 68)

(D) A list of species names

12. Using the penguins DataFrame, which line returns only the penguins heavier than 5000 g?

(A) `df.loc[df["body_mass_g"] > 5000]`

(B) `df["body_mass_g" > 5000]`

(C) `df[df["body_mass_g"] > 5000, :]`

(D) `df.filter(df["body_mass_g"] > 5000)`

13. Which line computes the average body mass per island?

(A) `df.groupby("island").mean("body_mass_g")`

(B) `df.groupby("body_mass_g")["island"].mean()`

(C) `df["island"].groupby(df["body_mass_g"]).mean()`

(D) `df.groupby("island")["body_mass_g"].mean()`

14. Which line returns the 5 penguins with the longest flippers?

(A) `df.head(5).sort_values("flipper_length_mm")`

(B) `df.sort_values("flipper_length_mm")[:5]`

(C) `df.sort_values("flipper_length_mm", ascending=False).head(5)`

(D) `df.sort_values("flipper_length_mm", ascending=False).tail(5)`

15. The team computes df.corr(). Based on the matrix below (columns abbreviated: bill_len = bill_length_mm, flipper = flipper_length_mm, body_mass = body_mass_g), which pair has the strongest positive correlation?

```
print(df.corr())
#              bill_len  bill_depth  flipper  body_mass
# bill_len        1.00       -0.24     0.66       0.60
# bill_depth     -0.24        1.00    -0.58      -0.47
# flipper         0.66       -0.58     1.00       0.87
# body_mass       0.60       -0.47     0.87       1.00
```

(A) flipper_length_mm & body_mass_g

(B) bill_length_mm & bill_depth_mm

(C) bill_length_mm & flipper_length_mm

(D) bill_depth_mm & body_mass_g

16. An adult income CSV uses "NA" and the sentinel -1 for missing values. Which line loads the file so both are recognized as missing?

(A) pd.read_csv("adult.csv", missing=["NA", -1])

(B) pd.read_csv("adult.csv", na_values=["NA", -1])

(C) pd.read_csv("adult.csv", na=["NA", -1])

(D) pd.read_csv("adult.csv", replace=[-1, np.nan])

17. The Titanic DataFrame has 177 rows with a missing age. Which line removes exactly those rows, keeping all other rows?

(A) df.dropna()

(B) df["age"].dropna()

(C) df.fillna(df["age"].mean())

(D) df.dropna(subset=["age"])

18. The passenger table has five rows with PassengerId = A1, A1, A2, A2, A3. After running df.drop_duplicates(subset=["PassengerId"], keep=False), how many rows remain?

(A) 5

(B) 4

(C) 3

(D) 1

19. Passenger names are stored in df["name"]. What does df["name"].str.contains("Lee") produce?

(A) A Series of True/False values, one per passenger

(B) A one-column DataFrame

(C) A list of the matching names

(D) A Series of counts

20. The income column holds ">50K" and "<=50K". Which line creates income_bool with True for ">50K" and False otherwise?

```
(A) df["income"].astype({">50K": True, "<=50K": False})
(B) df["income"].map({">50K": True, "<=50K": False})
(C) df["income"].str.upper()
(D) df["income"].value_counts()
```

Part 2 | Free Response

The California Housing Value Study. A data team at the California Department of Housing and Community Development is studying what drives median house values across the state. They use the public California Housing dataset (1990 census, StatLib archive), which contains one row per census block group. Follow the team through their workflow: data preparation in Python (Q21), numeric analysis with NumPy (Q22), exploratory analysis with pandas (Q23), and cleaning plus an AI-model comparison (Q24). All data excerpts needed are printed in each question.

Data used in this paper (excerpts simplified for printing): California housing (1990 census) - StatLib archive; NYC short-term rental listings - Inside Airbnb (insideairbnb.com); NYC yellow taxi trip records - NYC Taxi & Limousine Commission (open data); "Online Retail" transactions - UCI Machine Learning Repository; daily maximum temperatures - NOAA.

Q21 - Preparing the raw data in Python

The team received housing_raw.txt, whose lines look like this:

```
BG-06001,INLAND,180000,3.5,52
BG-06002,NEAR BAY,265000,4.8,42
BG-06003,INLAND,95000,2.4,18
BG-06004,<1H OCEAN,320000,5.2,29
BG-06005,INLAND,150000,3.9,45
```

Notes: each data line has exactly 5 comma-separated fields - block_group_id, ocean_proximity, median_house_value, median_income, housing_median_age; some lines may be blank and must be skipped.

(a) Write a function parse_line(line) that strips the line, splits it into the 5 fields, and returns the tuple (block_group_id, ocean_proximity, median_house_value, median_income, housing_median_age), where median_house_value is a float and housing_median_age is an int.

(b) Using parse_line and a list comprehension with a condition, write code that opens housing_raw.txt and builds blocks = [(block_group_id, ocean_proximity, median_house_value, median_income, housing_median_age), ...] containing only the non-blank lines.

(c) Using only Python (no NumPy/pandas), print the average median_house_value rounded to 2 decimals in the format "Average value: $...".

Q22 - Analyzing housing values with NumPy

The team loads the cleaned housing data into NumPy arrays (one block group per index):

```
values = np.array([180000.0, 265000.0, 95000.0, 320000.0, 0.0,
150000.0, 120000.0, 240000.0])
proximity = np.array(["INLAND", "NEAR BAY", "INLAND", "<1H OCEAN",
                      "INLAND", "NEAR BAY", "INLAND", "<1H OCEAN"])
```

The 0.0 in values is a suspected data error.

(a) Write one line (boolean mask or np.where) that treats values <= 0 as invalid and print the mean of the valid values (round to 2 decimals).

(b) Using boolean masking, print (i) how many block groups have a value above the mean, and (ii) the mean median_house_value of "INLAND" block groups.

(d) For a follow-up survey the team needs 3 block-group indices, chosen randomly but reproducibly. Using np.random.default_rng(seed=2026), write the code that picks 3 distinct indices, and explain in one sentence why the fixed seed matters.

Q23 - Exploratory analysis with pandas

The team reads the cleaned data into a DataFrame and prints a preview:

```
   block_group_id ocean_proximity  median_house_value  median_income
housing_median_age
0  BG-06001            INLAND              180000           3.5
52
1  BG-06002          NEAR BAY              265000           4.8
42
2  BG-06003            INLAND               95000           2.4
18
3  BG-06004         <1H OCEAN             320000           5.2
29
4  BG-06005            INLAND              150000           3.9
45
```

(a) Write the line of code that produces the following stats for the “median_house_value”, and briefly describe what you learn about its distribution.

```
count 1000.00   mean 206856.00   std 115396.00
min 14999.00   25% 119500.00   50% 179700.00   75% 264725.00   max
500001.00
```

(b) Write the line of code that calculates the mean “median house value” of all “INLAND” block groups.

(c) Based on the correlation matrix below, state which single factor is most strongly (positively) related to median_house_value, and say in plain words what that means.

```
                     median_house_value  median_income
housing_median_age  total_rooms
median_house_value                 1.00           0.69
-0.12         0.13
median_income                      0.69           1.00
0.01         0.20
housing_median_age                -0.12           0.01
1.00        -0.36
total_rooms                        0.13           0.20
-0.36         1.00
```

(d) Write the code that returns the 5 most expensive block groups, and state the most common ocean_proximity from the breakdown printed below.

```
df["ocean_proximity"].value_counts()
# <1H OCEAN    420
# INLAND       300
# NEAR OCEAN   130
# NEAR BAY     110
# ISLAND        40
```

Q24 - Cleaning, and comparing two AI assistants

The full dataset (housing.csv) has known issues: median_house_value can be empty strings "" or the sentinel -999; some block_group ids repeat; some ocean_proximity values have extra spaces (e.g. " INLAND ").

(a) Write the pd.read_csv line that loads housing.csv so that "" and -999 are both treated as missing, then write one line that reports how many missing values remain in each column.

(b) Write the line that removes rows whose median_house_value is missing, and the line that removes duplicate rows based on the block_group_id column, keeping the last occurrence.

(c) Write one line that cleans the ocean_proximity column with str.strip(), and one line (map or replace) that renames "NEAR BAY" to "Bay" and "NEAR OCEAN" to "Ocean", leaving all other values unchanged.

(d) After the cleaning steps in (a)-(c), the team asks two AI assistants the same question: "What is the mean median_house_value of <1H OCEAN block groups?" The two models' code and outputs are printed below. (i) Which model's answer is more trustworthy, and why? (ii) Identify which cleaning step(s) Model B skipped, and explain in what direction that omission moves the answer. (iii) Suggest one verification step the team should run before reporting either number.

```
Model A
df = pd.read_csv("housing.csv", na_values=["", "-999"])
df = df.dropna(subset=["median_house_value"])
df = df.drop_duplicates(subset=["block_group_id"], keep="last")
df["ocean_proximity"] = df["ocean_proximity"].str.strip()
m = df[df["ocean_proximity"] == "<1H OCEAN"]
print(m["median_house_value"].mean())   # 240244.0

Model B
df = pd.read_csv("housing.csv", na_values=["", "-999"])
df = df.dropna(subset=["median_house_value"])
m = df[df["ocean_proximity"] == "<1H OCEAN"]
print(m["median_house_value"].mean())   # 235800.0
```

Answer Sheet

Part 1 - Multiple Choice

1.	2.	3.	4.
5.	6.	7.	8.
9.	10.	11.	12.
13.	14.	15.	16.
17.	18.	19.	20.

Part 2 - Free Response

Q21

a)

---

b)

---

c)

Q22

a)

---

b)

c)

Q23

a)

b)

c)

d)

Q24

a)

b)

c)

d)